CSP-203: Software Tools & Technologies Lab



Plotting Tools: Matplotlib
(Lab Session – 16)

Dr. Anand M. Baswade
Indian Institute of Technology, Bhilai
anand@iitbhilai.ac.in

Contents

- ❑ Basics of Matplotlib
- ❑ Line Plot
- ❑ Barplot
- ❑ Histogram
- ❑ Scatter Plot
- ❑ Pie Chart

Introduction to Matplotlib

What is Matplotlib?

- Matplotlib is one of the most powerful tools for **data visualization** in Python.
- It is a **2D plotting library** that allows you to create a wide range of plots: line plots, bar charts, scatter plots, histograms, and more.
- Matplotlib was created to make Python plotting work similarly to MATLAB.

How to Install Matplotlib

The easiest way to install Matplotlib is using **pip**.

Open your terminal or command prompt and run:

```
pip install matplotlib
```

> **Note:** You may need to run the above command as an administrator.

If installation fails due to dependencies, run:

```
sudo apt-get update
sudo apt-get upgrade
```

Then try installing Matplotlib again.

Using Matplotlib in Your Script

To use Matplotlib in Python, import the `pyplot` module:

```
import matplotlib.pyplot as plt
```

- `matplotlib.pyplot` is a collection of **command-style functions** that make Matplotlib behave like MATLAB.
- Using `pyplot`, you can:
    - **Create a figure** and plotting area
    - **Plot lines and markers**
    - **Customize colors, labels, and styles**
- States like colors, labels, and axes are **preserved across function calls**, making plotting flexible.

Basic Workflow

1. **Choose the type of graph** you want to plot (e.g., line, bar, scatter).
2. **Display the graph** using:

```
plt.show()
```

Installing and Using Python on Linux

1. Check if Python is Already Installed

Open a terminal and run:

```
python3 --version
```

- This will display the installed Python version.
- If Python is not installed, the command will fail.

---

2. Update Package Lists

Before installing Python, update your package manager:

```
sudo apt-get update
sudo apt-get upgrade
```

---

3. Install Python

Install Python 3 using the package manager:

```
sudo apt-get install python3
```

- This installs Python 3 on your system.
- Optionally, install `pip` (Python package manager):

```
sudo apt-get install python3-pip
```

---

4. Verify Installation

Check that Python and pip were installed correctly:

```
python3 --version
pip3 --version
```

---

5. Running Python

You can run Python in two ways:

a) Interactive Mode

Open the Python shell:

```
python3
```

- You will see the `>>>` prompt.
- Type Python commands directly, e.g.:

```
print("Hello, Python!")
```

- Exit interactive mode by typing:

```
exit()
```

b) Running a Python Script

1. Create a file, e.g., `script.py`:

```
nano script.py
```

2. Write Python code inside the file:

```
print("Hello from script!")
```

3. Run the script:

```
python3 script.py
```

---

6. Using Python with Packages

After installing pip, you can install packages like Matplotlib, NumPy, and Pandas:

```
pip3 install matplotlib numpy pandas
```

```python
# Flower Data
flowers = ["Flower1", "Flower2", "Flower3", "Flower4", "Flower5",
           "Flower6", "Flower7", "Flower8", "Flower9", "Flower10"]

# Measurements (numeric data)
sepal_length = [5.1, 4.9, 5.8, 6.0, 5.4, 5.7, 6.3, 4.8, 5.5, 5.0]
sepal_width = [3.5, 3.0, 3.2, 3.4, 3.7, 3.1, 3.3, 3.6, 3.8, 3.2]
petal_length = [1.4, 1.5, 1.8, 2.0, 1.7, 1.9, 2.2, 1.3, 1.6, 1.4]
petal_width = [0.2, 0.2, 0.3, 0.4, 0.3, 0.3, 0.5, 0.2, 0.3, 0.2]

# Species for categorical plots
species = ["Setosa", "Setosa", "Versicolor", "Versicolor", "Setosa",
           "Versicolor", "Virginica", "Setosa", "Virginica", "Setosa"]

# Extra data for bar/horizontal bar plots (e.g., performance or sales)
flower_score = [85, 90, 78, 92, 88, 80, 95, 70, 89, 84]  # can represent "performance"
flower_sales = [150, 200, 170, 220, 180, 160, 210, 140, 190, 175]  # can represent "sales"

# Extra data for histogram demonstration (e.g., age or size)
flower_age = [1, 2, 3, 1, 2, 3, 4, 1, 2, 3]  # just small integer values for histogram

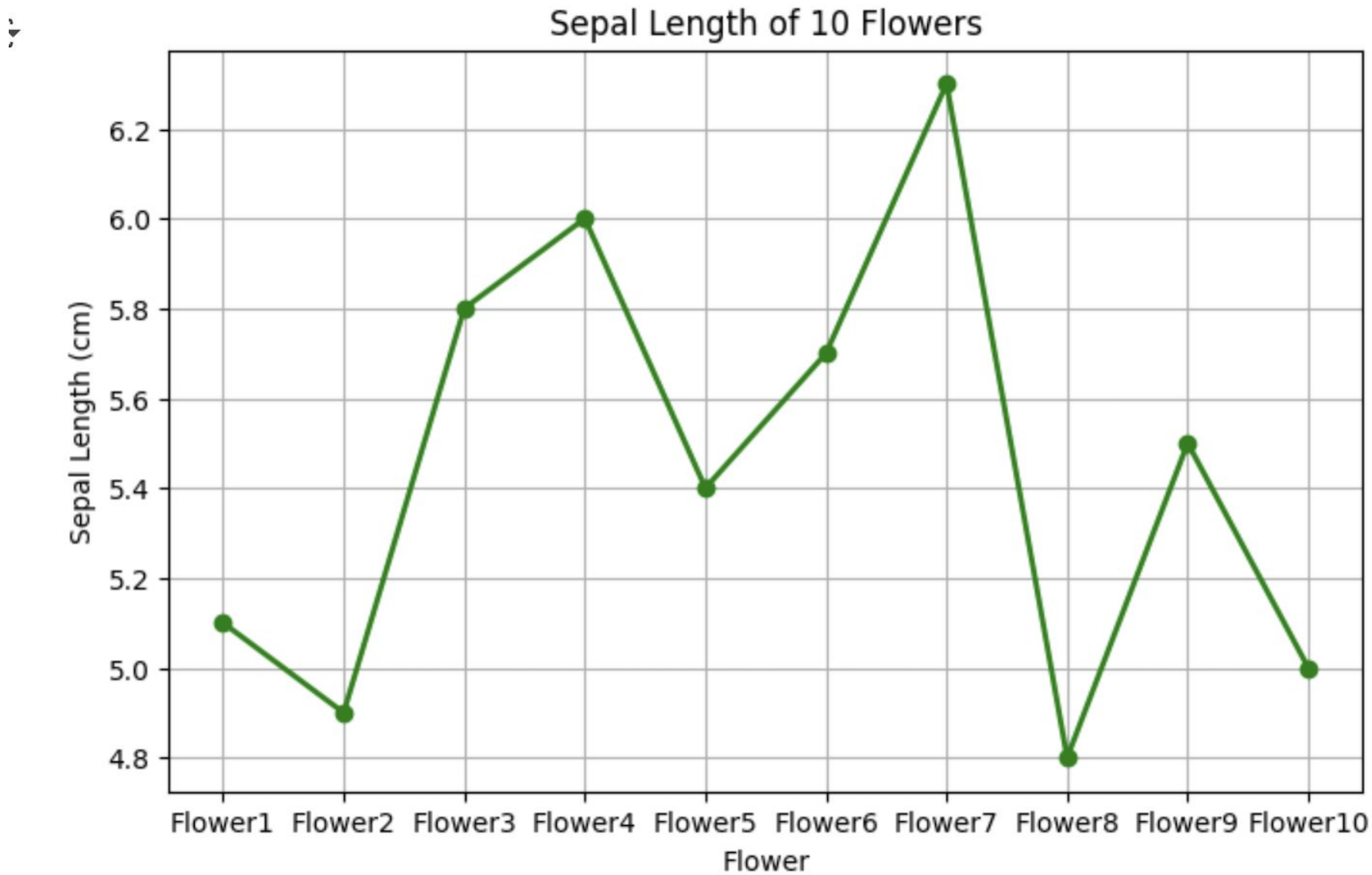
# Colors for categorical plots
species_colors = {'Setosa':'red', 'Versicolor':'green', 'Virginica':'blue'}
```

Line Plot: Sepal Length of Flowers

- **Purpose:** Visualize the sepal length of 10 flowers in a simple line graph.
- **Data Used:**
  - X-axis → `flowers` (categorical names of flowers)
  - Y-axis → `sepal_length` (numeric measurement in cm)
- **Customization:**
  - `marker='o'` → circle markers for each data point
  - `color='green'` → green line color
  - `linestyle='-'` → solid line connecting points
  - `linewidth=2` → thickness of the line
  - `plt.grid(True)` → adds gridlines for readability
  - `plt.figure(figsize=(8,5))` → sets the size of the plot
- **Steps in Code:**
  1. Import `matplotlib.pyplot` as `plt`.
  2. Define custom data lists for flowers and their sepal lengths.
  3. Use `plt.plot()` to create a line plot.
  4. Add title and axis labels with `plt.title()`, `plt.xlabel()`, `plt.ylabel()`.
  5. Display the plot with `plt.show()`.

```
# Custom dataset
flowers = ["Flower1", "Flower2", "Flower3", "Flower4", "Flower5",
           "Flower6", "Flower7", "Flower8", "Flower9", "Flower10"]
sepal_length = [5.1, 4.9, 5.8, 6.0, 5.4, 5.7, 6.3, 4.8, 5.5, 5.0]

# Plotting the line graph
plt.figure(figsize=(8,5))  # Set figure size
plt.plot(flowers, sepal_length, marker='o', color='green', linestyle='-', linewidth=2)
plt.title("Sepal Length of 10 Flowers")
plt.xlabel("Flower")
plt.ylabel("Sepal Length (cm)")
plt.grid(True)
plt.show()
```


Sepal Length of 10 Flowers
Sepal Length (cm)
6.2
6.0
5.8
5.6
5.4
5.2
5.0
4.8
Flower1
Flower2
Flower3
Flower4
Flower5
Flower6
Flower7
Flower8
Flower9
Flower10
Flower

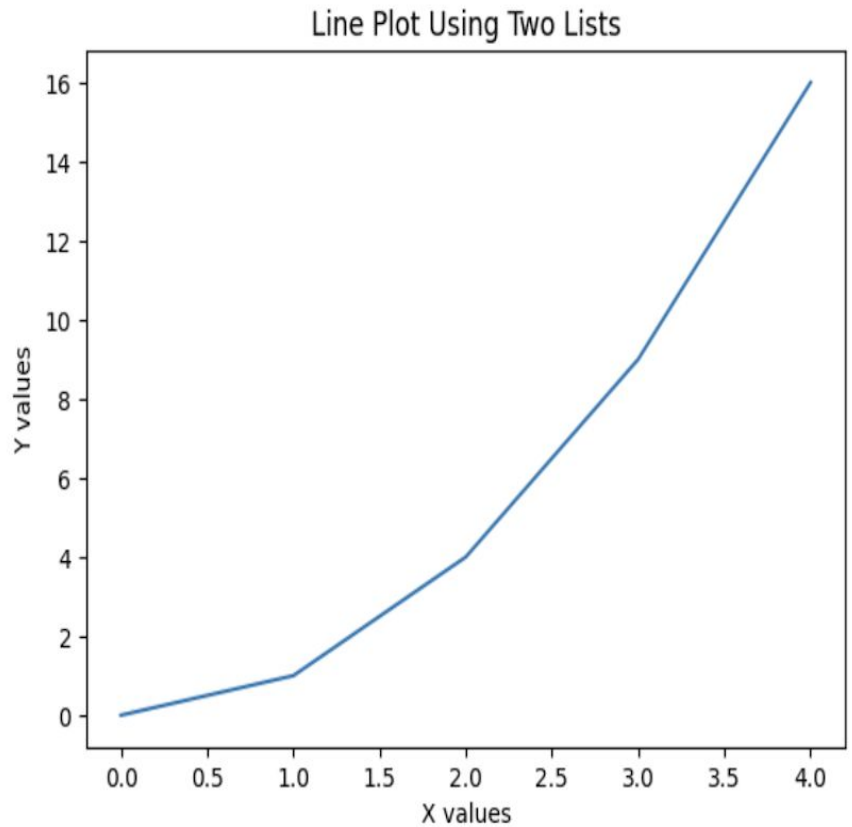
1️⃣ Line Plot Using Two Lists

15]
✓ 0s

```
# X and Y values
x_val = [0, 1, 2, 3, 4]
y_val = [0, 1, 4, 9, 16]

# Plotting the line graph
plt.plot(x_val, y_val)
plt.title("Line Plot Using Two Lists")
plt.xlabel("X values")
plt.ylabel("Y values")
plt.show()
```

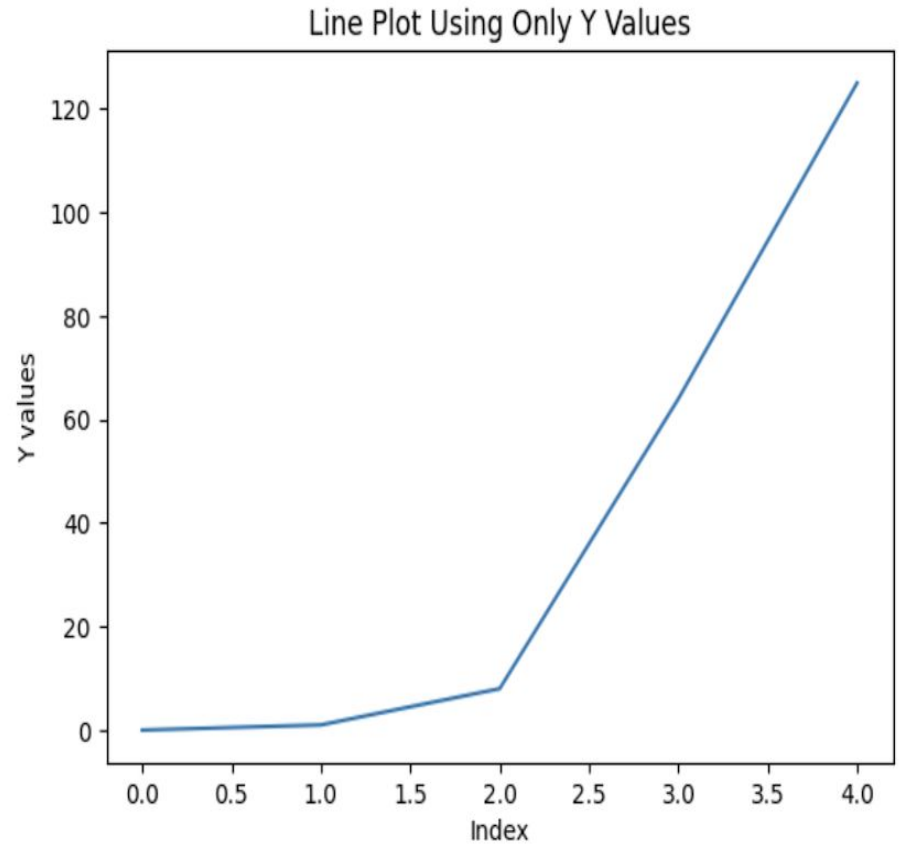
[ ]



2️⃣ Line Plot Using Only Y Values

```
# Only Y values are provided
y_val = [0, 1, 8, 64, 125]

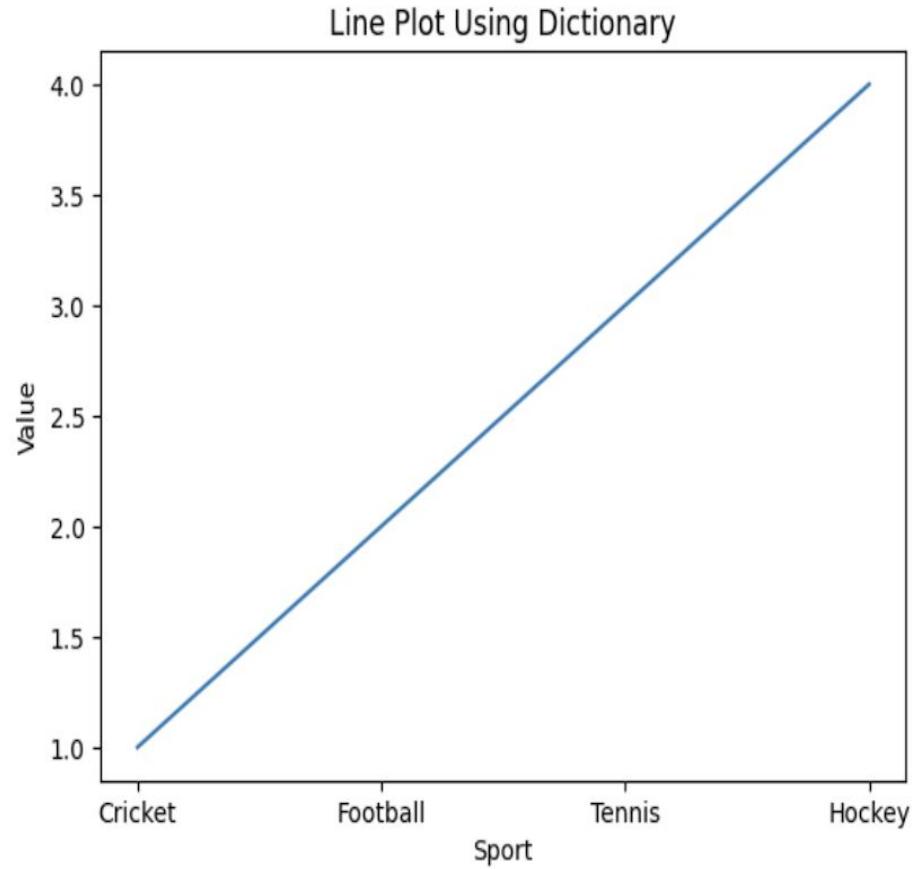
# Matplotlib automatically generates X values: 0,1,2,...
plt.plot(y_val)
plt.title("Line Plot Using Only Y Values")
plt.xlabel("Index")
plt.ylabel("Y values")
plt.show()
```



3️⃣ Line Plot Using a Dictionary

```
# Dictionary with categories as keys and numeric values
data = {"Cricket": 1, "Football": 2, "Tennis": 3, "Hockey": 4}

# Plot using dictionary keys and values
plt.plot(list(data.keys()), list(data.values()))
plt.title("Line Plot Using Dictionary")
plt.xlabel("Sport")
plt.ylabel("Value")
plt.show()
```



4️⃣ Line Plot with Customizations and Text

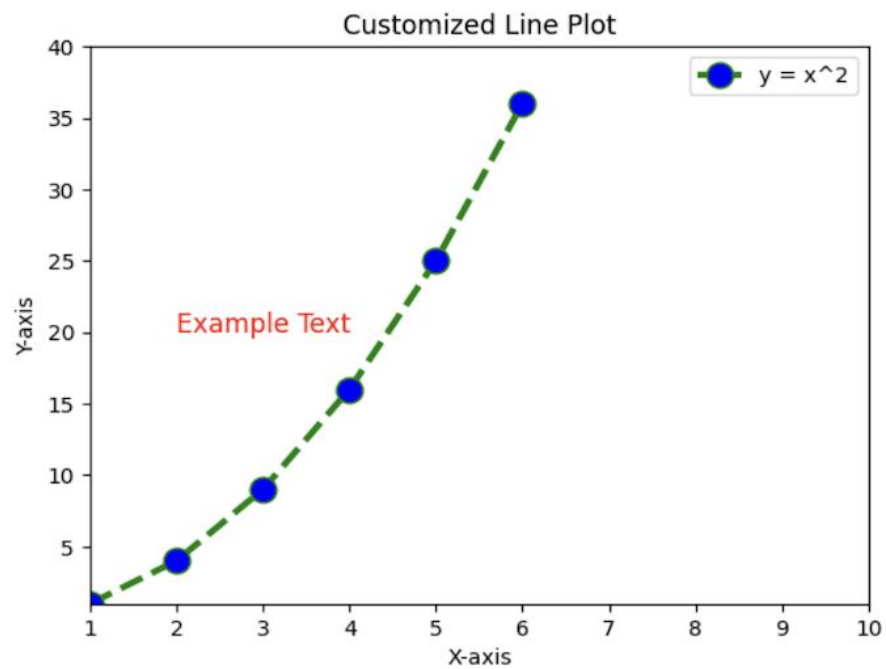
```
# Custom X and Y values
x = [1, 2, 3, 4, 5, 6]
y = [1, 4, 9, 16, 25, 36]

# Plot with customization
plt.plot(x, y, color='green', linestyle='dashed', linewidth=3,
         marker='o', markerfacecolor='blue', markersize=12)

# Set axis limits
plt.xlim(1, 10)
plt.ylim(1, 40)

# Labels, title, and text
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.title("Customized Line Plot")
plt.text(2, 20, "Example Text", fontsize=12, color='red')

# Show legend
plt.legend(["y = x^2"])
plt.show()
```



✔ Vertical and Horizontal Bar Plots

1️⃣ We have a custom dataset of 10 flowers with their `performance scores` and `sales units`.

2️⃣ **Vertical Bar Plot**:

- `plt.bar()` is used for a vertical bar chart.
- X-axis shows flower names.
- Y-axis shows scores.
- Gridlines on Y-axis improve readability.

3️⃣ **Horizontal Bar Plot**:

- `plt.barh()` is used for a horizontal bar chart.
- Y-axis shows flower names.
- X-axis shows units sold.
- Gridlines on X-axis improve readability.

4️⃣ Colors are customized for better visualization:

- Vertical: skyblue
- Horizontal: lightgreen

5️⃣ Titles and axis labels are added for clarity.

```python
import matplotlib.pyplot as plt

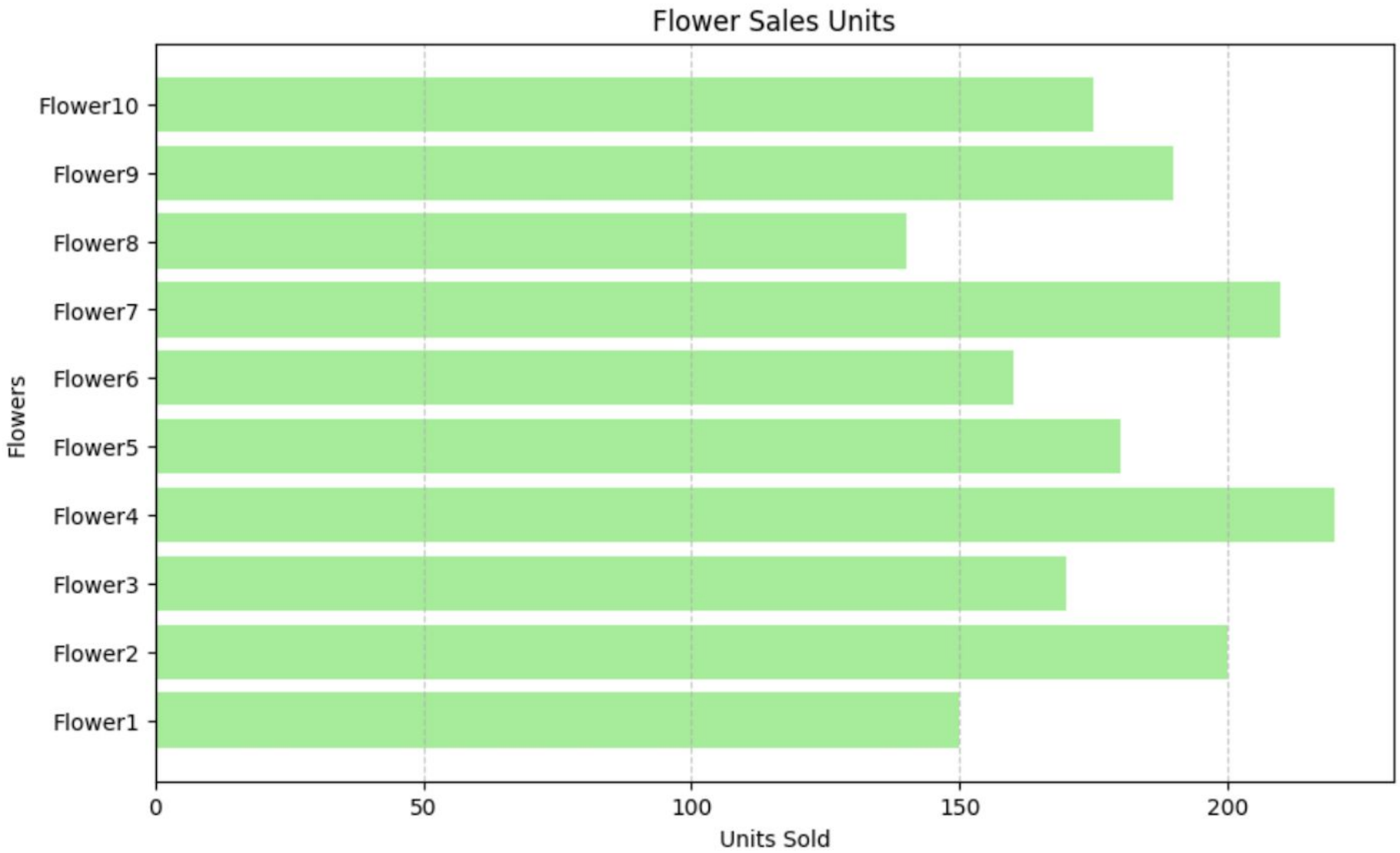
# Flowers and scores
flowers = ["Flower1", "Flower2", "Flower3", "Flower4", "Flower5",
           "Flower6", "Flower7", "Flower8", "Flower9", "Flower10"]
flower_score = [85, 90, 78, 92, 88, 80, 95, 70, 89, 84]

# Vertical Bar Plot
plt.figure(figsize=(10,6))
plt.bar(flowers, flower_score, color='skyblue')
plt.title("Flower Performance Scores")
plt.xlabel("Flowers")
plt.ylabel("Score")
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```


Flower Performance Scores
Score
0
20
40
60
80
Flower1
Flower2
Flower3
Flower4
Flower5
Flower6
Flower7
Flower8
Flower9
Flower10
Flowers

```
# Flower sales
flower_sales = [150, 200, 170, 220, 180, 160, 210, 140, 190, 175]

# Horizontal Bar Plot
plt.figure(figsize=(10,6))
plt.barh(flowers, flower_sales, color='lightgreen')
plt.title("Flower Sales Units")
plt.xlabel("Units Sold")
plt.ylabel("Flowers")
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.show()
```


Flower Sales Units
Flowers
Flower10
Flower9
Flower8
Flower7
Flower6
Flower5
Flower4
Flower3
Flower2
Flower1
0
50
100
150
200
Units Sold

Histogram of Flower Scores

1️⃣ We have a custom dataset of 10 flowers with their performance scores.

2️⃣ **Histogram**:

- `plt.hist()` is used to plot the frequency distribution of the scores.
- `bins=5` divides the scores into 5 intervals (70–76, 76–82, etc.).
- `range=(70,100)` sets the minimum and maximum values for the histogram.

3️⃣ **Visual Customization**:

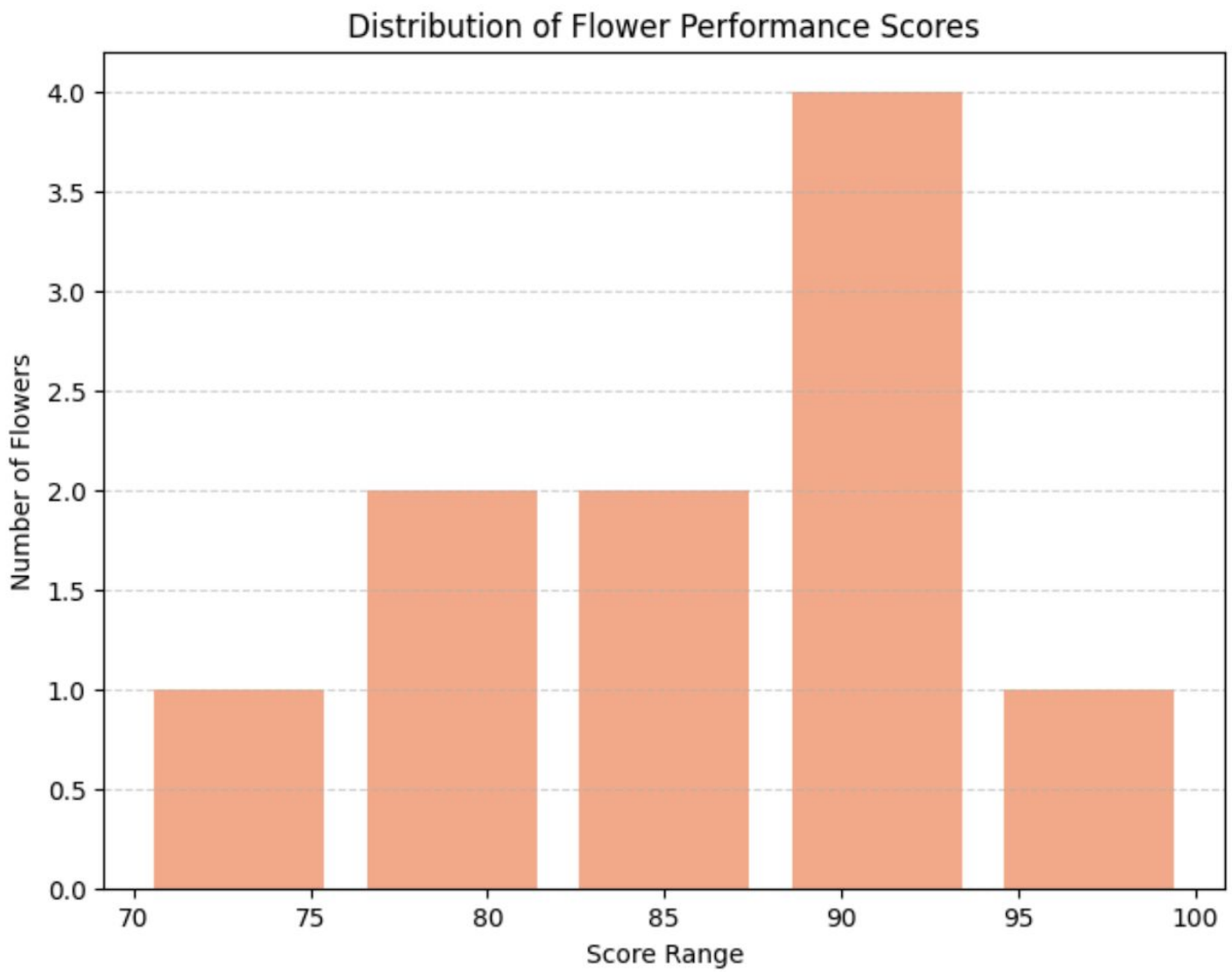
- `color='coral'` gives a distinct color to bars.
- `rwidth=0.8` makes bars occupy 80% of the bin width (spacing between bars).
- `alpha=0.7` sets transparency for better visualization.
- Gridlines along Y-axis improve readability.

4️⃣ **Labels and Title**:

- X-axis: "Score Range"
- Y-axis: "Number of Flowers"
- Title: "Distribution of Flower Performance Scores"

```
# Flower performance scores
flower_score = [85, 90, 78, 92, 88, 80, 95, 70, 89, 84]

# Plotting Histogram
plt.figure(figsize=(8,6))
plt.hist(flower_score, bins=5, range=(70, 100), color='coral',
         histtype='bar', rwidth=0.8, alpha=0.7)
plt.xlabel("Score Range")
plt.ylabel("Number of Flowers")
plt.title("Distribution of Flower Performance Scores")
plt.grid(axis='y', linestyle='--', alpha=0.6)
plt.show()
```

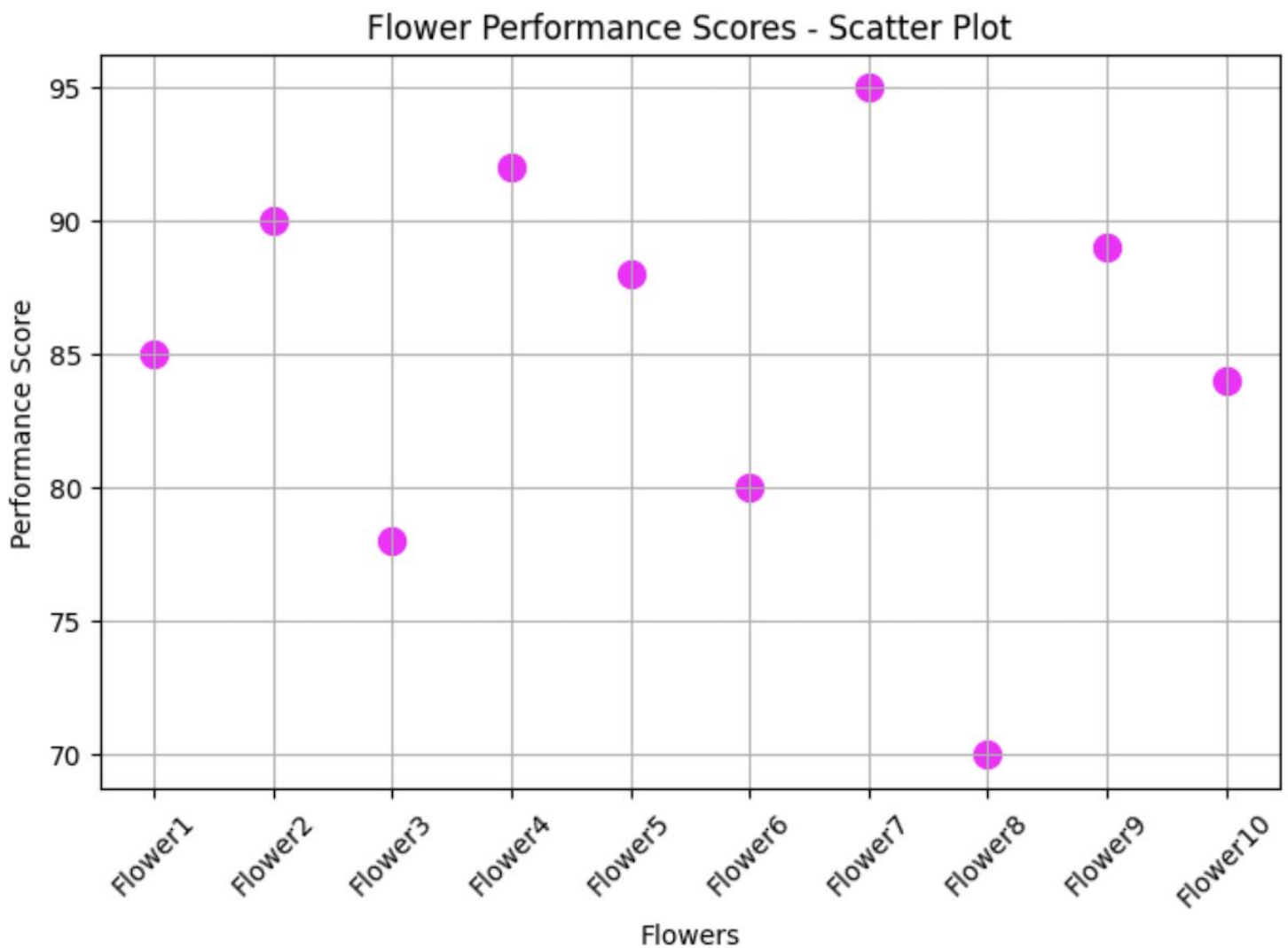

Distribution of Flower Performance Scores
Number of Flowers
Score Range
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
70
75
80
85
90
95
100

1️⃣ Single Scatter Plot

- `plt.scatter()` plots individual points.
- X-axis: Index of flowers (0–9), Y-axis: performance scores.
- `s=100` sets the size of points.
- `color='magenta'` gives distinct color.
- `plt.xticks()` labels each point with flower names.
- `plt.grid(True)` adds gridlines for readability.

```
# ---------- Scatter Plot (Single) ----------
# Using flower performance scores as y-values
flowers = ["Flower1", "Flower2", "Flower3", "Flower4", "Flower5",
           "Flower6", "Flower7", "Flower8", "Flower9", "Flower10"]
flower_score = [85, 90, 78, 92, 88, 80, 95, 70, 89, 84]

plt.figure(figsize=(8,5))
plt.scatter(range(len(flower_score)), flower_score, s=100, color='magenta')
plt.xticks(range(len(flowers)), flowers, rotation=45)
plt.xlabel("Flowers")
plt.ylabel("Performance Score")
plt.title("Flower Performance Scores - Scatter Plot")
plt.grid(True)
plt.show()
```

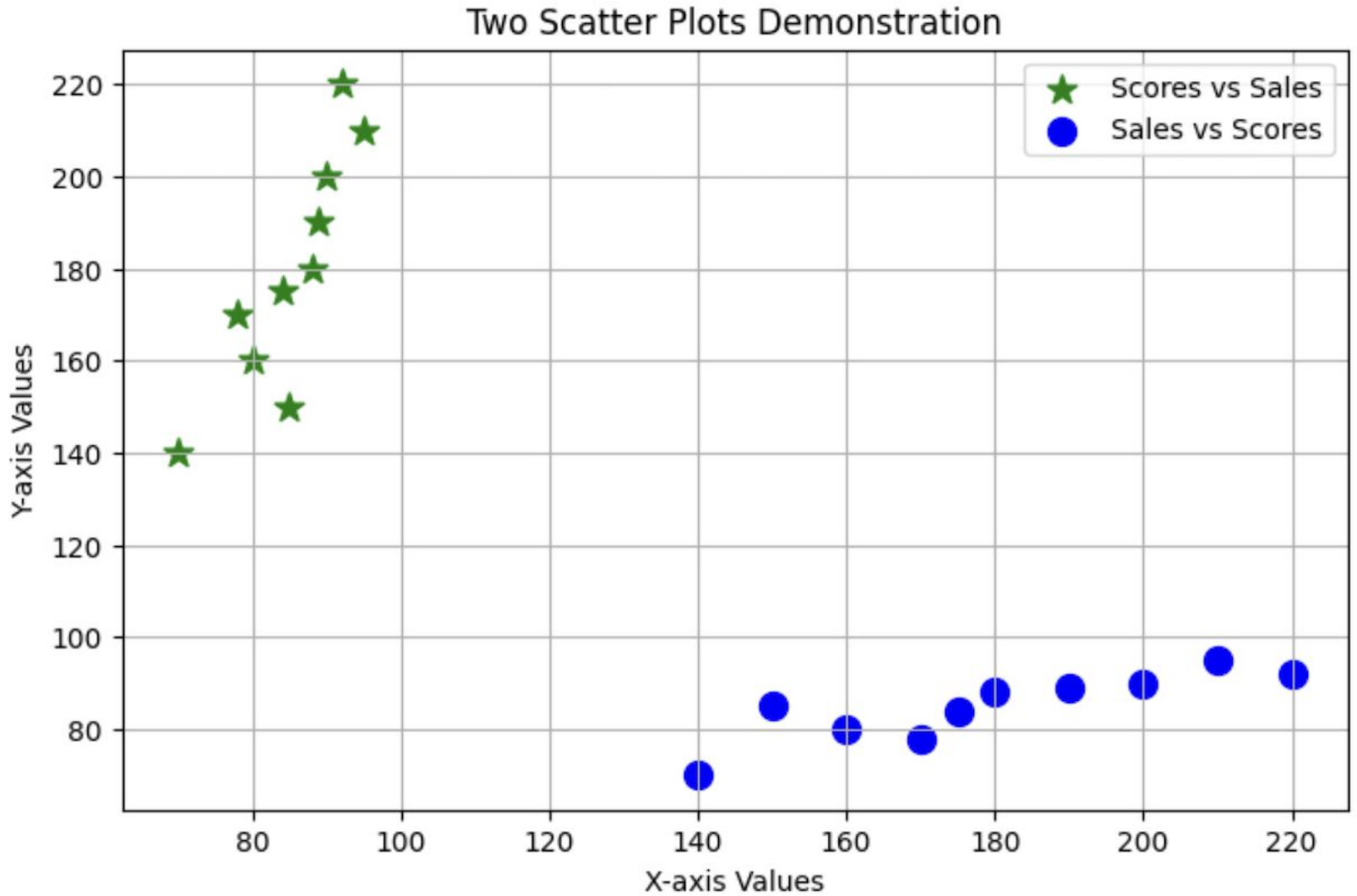

Flower Performance Scores - Scatter Plot
Performance Score
95
90
85
80
75
70
Flower1
Flower2
Flower3
Flower4
Flower5
Flower6
Flower7
Flower8
Flower9
Flower10
Flowers

2️⃣ Two Scatter Plots

- Demonstrates multiple scatter plots in one figure.
- First plot: Scores vs Sales with green stars (*).
- Second plot: Sales vs Scores with blue circles (o).
- `plt.legend()` distinguishes the two sets.
- Custom markers visually differentiate the data points.

```
# ---------- Scatter Plot (Two Sets) ----------
# Using flower scores vs. flower sales
flower_sales = [150, 200, 170, 220, 180, 160, 210, 140, 190, 175]

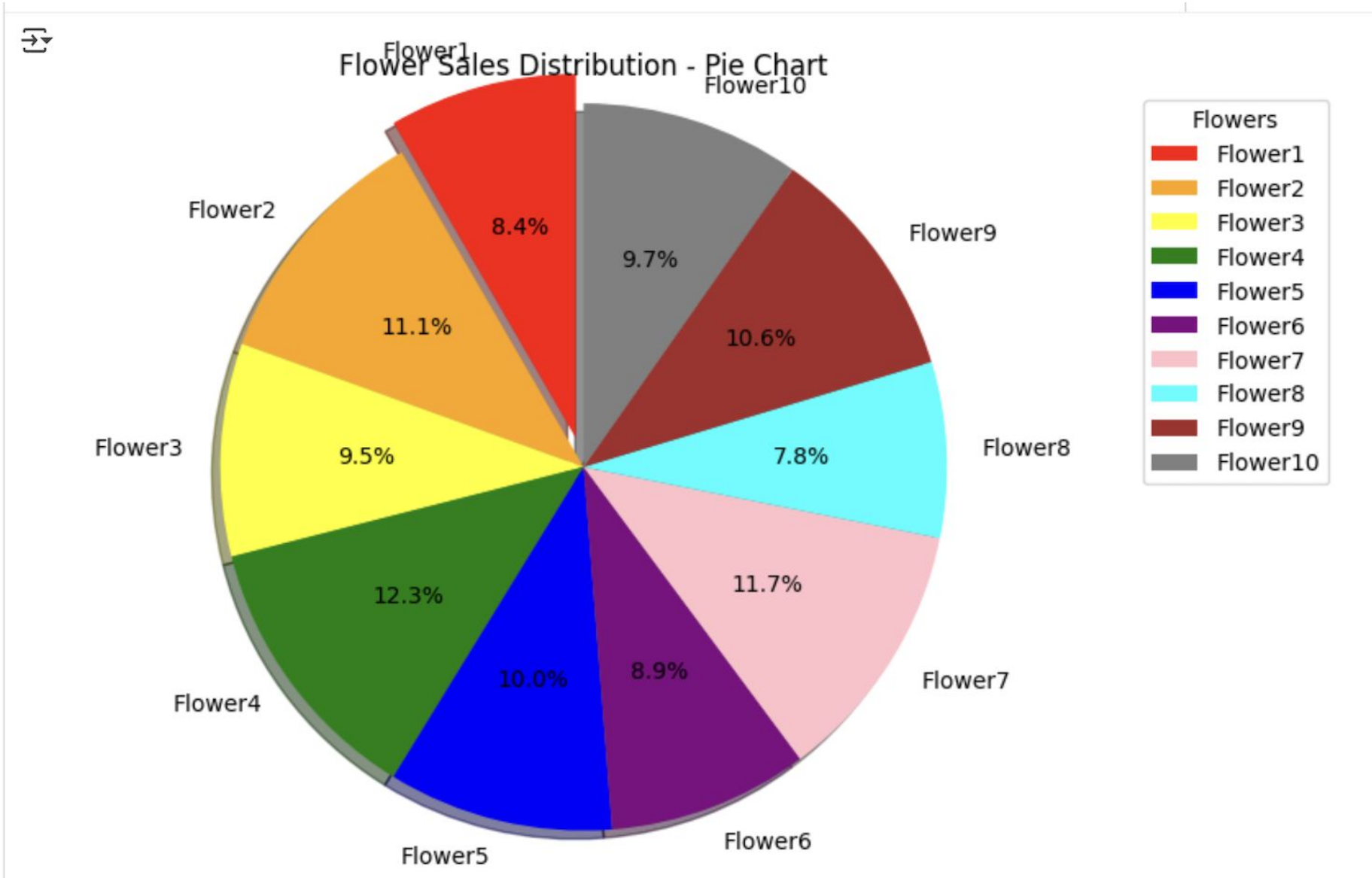
plt.figure(figsize=(8,5))
plt.scatter(flower_score, flower_sales, label="Scores vs Sales", color='green', marker='*', s=120)
plt.scatter(flower_sales, flower_score, label="Sales vs Scores", color='blue', marker='o', s=100)
plt.xlabel("X-axis Values")
plt.ylabel("Y-axis Values")
plt.title("Two Scatter Plots Demonstration")
plt.legend()
plt.grid(True)
plt.show()
```


Two Scatter Plots Demonstration
Scores vs Sales
Sales vs Scores
Y-axis Values
X-axis Values
220
200
180
160
140
120
100
80
80
100
120
140
160
180
200
220

3️⃣ Pie Chart

- `plt.pie()` creates a pie chart for flower sales.
- `labels=flowers` names each slice.
- `colors` assigns a unique color to each slice.
- `explode` emphasizes a particular slice (first flower in this case).
- `autopct='%1.1f%%'` shows percentage values on slices.
- `shadow=True` adds depth to the chart.
- `plt.legend()` adds a legend to identify slices.

```
# ---------- Pie Chart ----------
# Using flower sales distribution as proportions
plt.figure(figsize=(6,6))
colors = ['red', 'orange', 'yellow', 'green', 'blue', 'purple', 'pink', 'cyan', 'brown', 'gray']
explode = [0.1 if i == 0 else 0 for i in range(len(flowers))]  # Highlight first slice
plt.pie(flower_sales, labels=flowers, colors=colors, startangle=90, shadow=True,
        explode=explode, autopct='%1.1f%%', radius=1.2)
plt.title("Flower Sales Distribution - Pie Chart")
plt.legend(title="Flowers", bbox_to_anchor=(1,0,0.5,1))
plt.show()
```


Flower Sales Distribution - Pie Chart
Flower1
Flower10
Flower2
Flower9
Flower3
Flower8
Flower4
Flower7
Flower5
Flower6
8.4%
11.1%
9.5%
12.3%
10.0%
8.9%
11.7%
7.8%
10.6%
9.7%
Flowers
Flower1
Flower2
Flower3
Flower4
Flower5
Flower6
Flower7
Flower8
Flower9
Flower10

1 Different Markers

Symbol	Marker	Description
'o'	Circle	Default point marker
'.'	Point	Small dot
','	Pixel	Very small dot
'x'	X	Cross
'+'	Plus	Plus sign
'*'	Star	Asterisk
's'	Square	Square marker
'D'	Diamond	Regular diamond
'd'	Thin diamond	Narrow diamond
'v'	Triangle down	Triangle pointing down
'^'	Triangle up	Triangle pointing up
'<'	Triangle left	Triangle pointing left
'>'	Triangle right	Triangle pointing right
'h'	Hexagon 1	Hexagon marker
'H'	Hexagon 2	Hexagon marker
'p'	Pentagon	Pentagon marker

2 Color Codes

Code	Color	Description
'b'	Blue	Blue
'g'	Green	Green
'r'	Red	Red
'c'	Cyan	Blue-green
'm'	Magenta	Pink-purple
'y'	Yellow	Yellow
'k'	Black	Black
'w'	White	Background

3 Color Names

Name	Color Display
blue	Blue
green	Green
red	Red
cyan	Cyan
magenta	Magenta
yellow	Yellow
black	Black
white	White
orange	Orange
purple	Purple
brown	Brown
pink	Pink
gray	Gray
olive	Greenish-yellow
navy	Dark blue
gold	Gold
lime	Bright green
teal	Blue-green
coral	Orange-pink

4 Hexadecimal Color Codes

Hex Code	Color
#FF0000	Red
#00FF00	Green
#0000FF	Blue
#FFFF00	Yellow
#00FFFF	Cyan
#FF00FF	Magenta
#FFA500	Orange
#800080	Purple
#808080	Gray
#000000	Black
#FFFFFF	White

---

5 Line Styles

Code	Description	Example
'-'	Solid line	————
'--'	Dashed line	— — — —
'-.'	Dash-dot line	— · — · —
':'	Dotted line	·····
'' or None	No line (markers only)	• • • •

Matplotlib Pie Chart - Legend Placement Demo

---

import matplotlib.pyplot as plt

Data

labels = ['Python', 'C++', 'Java', 'Go'] sizes = [45, 25, 20, 10]

1️⃣ Legend outside right

plt.figure(figsize=(5, 4)) plt.pie(sizes, labels=None, autopct='%1.1f%%') plt.legend(labels, loc='upper left', bbox_to_anchor=(1, 1))
plt.title("Legend Outside Right (bbox_to_anchor=(1, 1))") plt.show()

2️⃣ Legend below center

plt.figure(figsize=(5, 4)) plt.pie(sizes, labels=None, autopct='%1.1f%%') plt.legend(labels, loc='upper center', bbox_to_anchor=(0.5, -0.05))
plt.title("Legend Below Chart (bbox_to_anchor=(0.5, -0.05))") plt.show()

3️⃣ Legend to the left

plt.figure(figsize=(5, 4)) plt.pie(sizes, labels=None, autopct='%1.1f%%') plt.legend(labels, loc='upper right', bbox_to_anchor=(-0.1, 1))
plt.title("Legend on Left (bbox_to_anchor=(-0.1, 1))") plt.show()

4️⃣ Legend above chart

plt.figure(figsize=(5, 4)) plt.pie(sizes, labels=None, autopct='%1.1f%%') plt.legend(labels, loc='lower center', bbox_to_anchor=(0.5, 1.1))
plt.title("Legend Above Chart (bbox_to_anchor=(0.5, 1.1))") plt.show()

```python
# ------------------------------------------
# Matplotlib Pie Chart – Legend Placement Demo
# ------------------------------------------
import matplotlib.pyplot as plt

# Data
labels = ['Python', 'C++', 'Java', 'Go']
sizes = [45, 25, 20, 10]

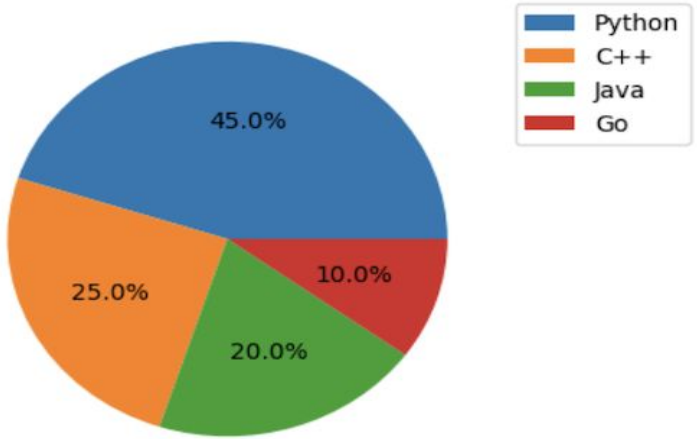
# 1️⃣ Legend outside right
plt.figure(figsize=(5, 4))
plt.pie(sizes, labels=None, autopct='%1.1f%%')
plt.legend(labels, loc='upper left', bbox_to_anchor=(1, 1))
plt.title("Legend Outside Right (bbox_to_anchor=(1, 1))")
plt.show()

# 2️⃣ Legend below center
plt.figure(figsize=(5, 4))
plt.pie(sizes, labels=None, autopct='%1.1f%%')
plt.legend(labels, loc='upper center', bbox_to_anchor=(0.5, -0.05))
plt.title("Legend Below Chart (bbox_to_anchor=(0.5, -0.05))")
plt.show()

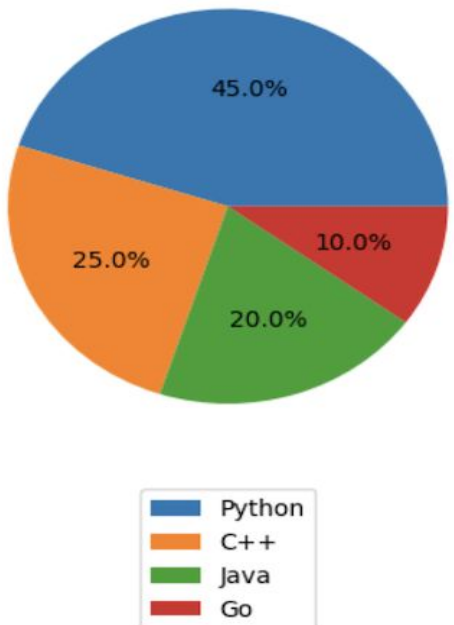
# 3️⃣ Legend to the left
plt.figure(figsize=(5, 4))
plt.pie(sizes, labels=None, autopct='%1.1f%%')
plt.legend(labels, loc='upper right', bbox_to_anchor=(-0.1, 1))
plt.title("Legend on Left (bbox_to_anchor=(-0.1, 1))")
plt.show()

# 4️⃣ Legend above chart
plt.figure(figsize=(5, 4))
plt.pie(sizes, labels=None, autopct='%1.1f%%')
plt.legend(labels, loc='lower center', bbox_to_anchor=(0.5, 1.1))
plt.title("Legend Above Chart (bbox_to_anchor=(0.5, 1.1))")
plt.show()
```

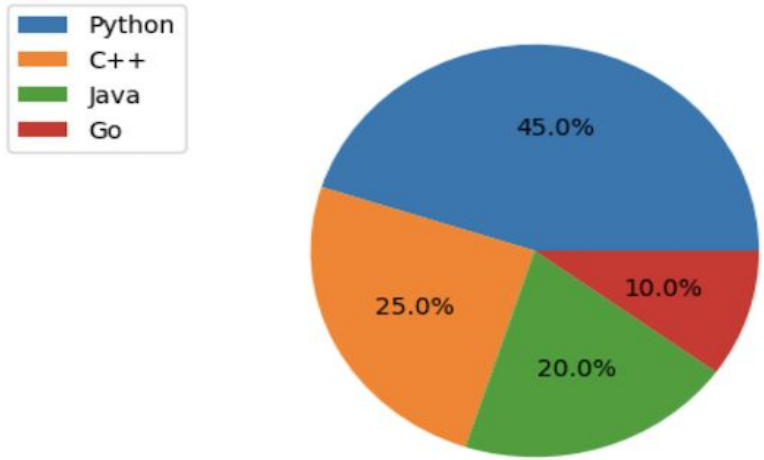
Legend Outside Right (bbox_to_anchor=(1, 1))

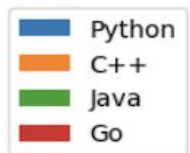


Legend Below Chart (bbox_to_anchor=(0.5, -0.05))

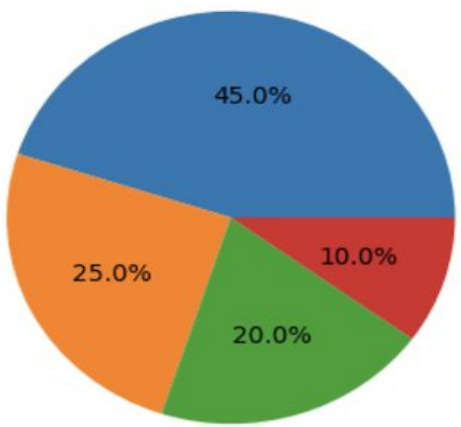


Legend on Left (bbox_to_anchor=(-0.1, 1))





Legend Above Chart (bbox_to_anchor=(0.5, 1.1))





Next Topic :Matplotlib Advanced